Simulation of Robotic Systems Lab 05

Lätitzia Brandt

June 27, 2025

Task 1

In the CopelliaSim scene file Car_Cuboid.ttt you see a blue rigid body (cuboid) and 4 dummies in red. The dummies are used as control points of a cubic Bezier spline. You should implement a cubic spline from these control points and move the cuboid along the spline curve (interpolating position and orientation).

Hint: The spline curve is two dimensional (x- and y-coordinate). For the orientation you must evaluate the tangent along the spline curve The program must be flexible in such a way, that a shift of the control points will result in a different motion of the cuboid.
You can use the python script bezier_path.py to build a bezier spline (and its derivative). See also its main function.

Preparation

Since the prep specified "Prepare a written program idea to implement the task", there is not much to show.

However, the initial idea for the final code was as follows:

1) get control point handles
2) create instance of class BezierSpline2D
3) set number of points for path
4) calculate the path

which was implemented in an initial draft of the code that was subsequently refined for ease of use by e.g. adding a listener for key presses.

Implementation

Calculating the bezier curve

```
    # getting the handles for the control points
    for i in range(nbr):
        cp[i] = sim.getObjectHandle('./CP[' + str(i) + ']')

    # set points
    for i in range(nbr):
        p[i] = sim.getObjectPosition(cp[i], -1)

    # creating BezierSpline2D instance
    bezier = BezierSpline2D(p[0], p[1], p[2], p[3])

    # setting number of points in the path
    nbr_path_pts = bezier.number_pts = 20

    # calculating bezier path
    path, x, y, yaw = bezier.calc_bezier_path()
```

Moving the car along the curve

```
for i in range(nbr_path_pts):
    time.sleep(0.04)
    sim.setObjectPosition(cuboid, [x[i], y[i], z])
    sim.setObjectOrientation(cuboid, [0, 0, yaw[i]])
```

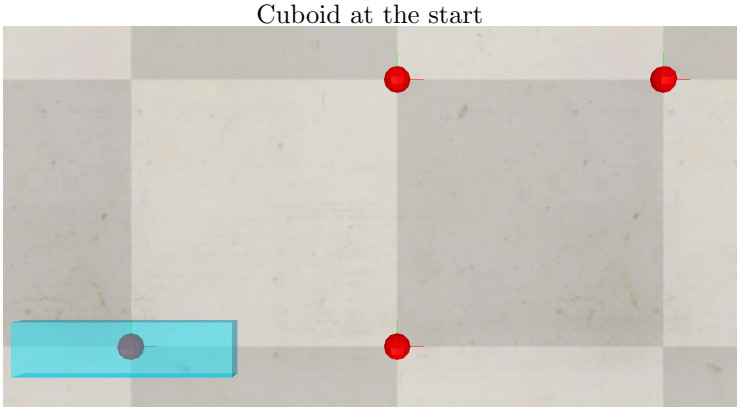
Explanation

To calculate the bezier curve we first must aquire all necessary parameters.
We start by getting the object handles of the control points that are part of the CoppeliaSim scene. We save their position in an array p[] and then create a BezierSpline2D object by passing the positions as parameters.
We also set the path points that make up the curve to 20. Any less and it might be too jagged, making the movement jerky, whereas more doesn't make any visual difference.
At last we calculate the bezier curve using the calc_bezier_path() and save the outputs in six variables of which we use x, y and yaw.

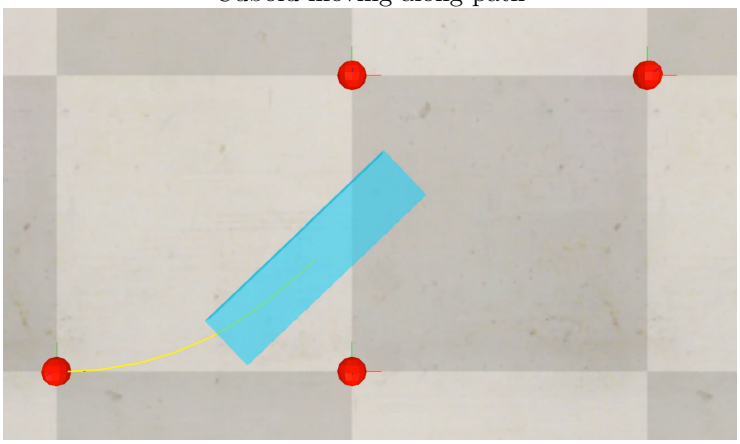
To move the cuboid, we set its position and orientation inside a for loop. Between every loop, the program sleeps a bit to slow the curve's movement.
It loops until all path points have been processed.

Screenshots

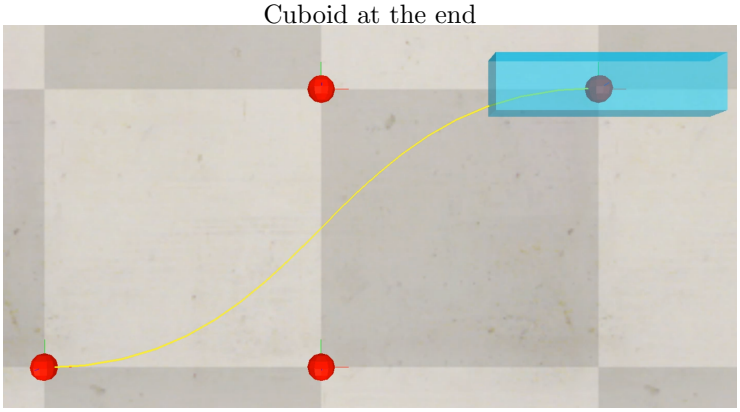
Cuboid at the start



Cuboid moving along path



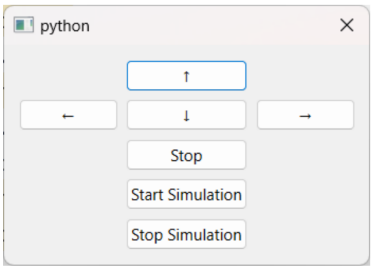
Cuboid at the end



Task 2

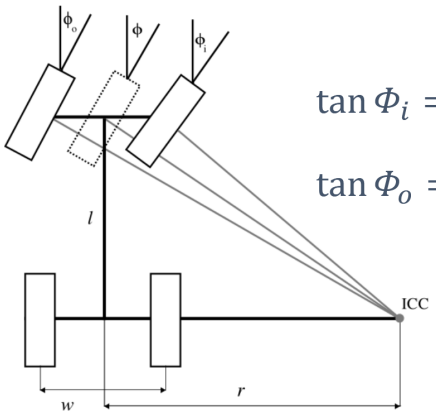
In the CopelliaSim scene file Wheel_Car.ttt you see a car with red front wheels. You should implement a GUI, see figure, which can drive the shown car in the following way:

- arrow up: increase speed
- arrow down: decrease speed
- arrow left: increase steering angle
- arrow right: decrease steering angle
- Stop: stop motion (setting joint velocities of left and right wheel to 0)



From the car model you should derive the important constructive measures like wheel base, wheel radius, and the distance between left and right wheels. The maximum steering is +/- 30 degrees. These parameters are necessary for the Ackermann steering of this car.

Ackermann steering formula:



$$d := \frac{w}{2}$$

$$\tan \Phi_i = \frac{l}{r-d} = \frac{l}{\frac{l}{\tan \Phi} - d} = \frac{l \tan \Phi}{l - d \tan \Phi}$$

$$\tan \Phi_o = \frac{l}{r+d} = \frac{l}{\frac{l}{\tan \Phi} + d} = \frac{l \tan \Phi}{l + d \tan \Phi}$$

l: wheel base: distance between front and rear wheels
r: turning radius (has a relation to the steering angle)
w: track width: distance between left and right wheels

Implementation

Important global variables

```
# max steering angle
max_inner_angle = math.radians(30)
max_speed = 20
```

Function for calculating the turning angle

```
def turn(angle, wB, tW, lwheel, rwheel):
    # wB = wheel base
    # tW = tire width, already halved.
    # Formula : angle = atan2((wB*math.tan(angle))/(wB+-0.5*tW*math.tan(
        angle)))
    d = 0.5 * tW
    left_angle = math.atan2((wB * math.tan(angle)), (wB - d * math.tan(
        angle)))
    right_angle = math.atan2((wB * math.tan(angle)), (wB + d * math.tan(
        angle)))
    # print("Left Angle : ", left_angle)
    # print("Right Angle : ", right_angle)
    return left_angle, right_angle
```

Class for the GUI - snippet shows only wheel movement

```
class Form(QDialog):

    [...]

    def goLeft(self):
        print("Going left")

        # calc the inner angle
        self.innerangle += math.radians(self.anglechange)
        print("Inner angle: ", self.innerangle)
        # clamp the angle
        self.innerangle = min(self.innerangle, max_inner_angle)

        # get angles for left and right wheel
        lwheel_angle = sim.getJointPosition(self.fl_steer)
        rwheel_angle = sim.getJointPosition(self.fr_steer)
        print("Left Wheel : ", lwheel_angle)
        print("Right Wheel : ", rwheel_angle)

        # final wheel angle for both wheels
        wheel_angles = turn(self.innerangle, self.length, self.diameter,
            lwheel_angle, rwheel_angle)

        sim.setJointTargetPosition(self.fl_steer, wheel_angles[0])
        sim.setJointTargetPosition(self.fr_steer, wheel_angles[1])
```

```
    def goRight(self):
        print("Going right")

        self.innerangle -= math.radians(self.anglechange)
        print("Inner angle: ", self.innerangle)
        self.innerangle = max(self.innerangle, -max_inner_angle)

        lwheel_angle = -(sim.getJointPosition(self.fl_steer))
        rwheel_angle = -(sim.getJointPosition(self.fr_steer))
        print("Left Wheel : ", lwheel_angle)
        print("Right Wheel : ", rwheel_angle)

        wheel_angles = turn(self.innerangle, self.length, self.diameter,
            lwheel_angle, rwheel_angle)
        print("Wheel angles : [Left - ", math.degrees(wheel_angles[0]), ",
            Right - ", math.degrees(wheel_angles[1]),
              "]")
        sim.setJointTargetPosition(self.fl_steer, wheel_angles[0])
        sim.setJointTargetPosition(self.fr_steer, wheel_angles[1])

    def goForwards(self):
        print("Going forwards")
        # increase speed by 1
        self.speed += 1
        # cap at max speed
        self.speed = min(self.speed, max_speed)
        sim.setJointTargetVelocity(self.fl_drive, self.speed)
        sim.setJointTargetVelocity(self.fr_drive, self.speed)

    def goBackwards(self):
        print("Going backwards")
        self.speed -= 1
        self.speed = max(self.speed, -max_speed)
        sim.setJointTargetVelocity(self.fl_drive, self.speed)
        sim.setJointTargetVelocity(self.fr_drive, self.speed)

  [...]

    def stop(self):
        self.speed = 0
        sim.setJointTargetVelocity(self.fl_drive, 0.0)
        sim.setJointTargetVelocity(self.fr_drive, 0.0)

    [...]
```

Main

```
        print('Program started')
    client = RemoteAPIClient()
    # Get objects
    sim = client.getObject('sim')
```

```
# The wheels
fl_drive = sim.getObjectHandle('./motorLeft')
fr_drive = sim.getObjectHandle('./motorRight')
fl_steer = sim.getObjectHandle('./steeringLeft')
fr_steer = sim.getObjectHandle('./steeringRight')
length = 2.5772  # From construction.
width = 2*0.75515  # From construction

app = QApplication(sys.argv)
form = Form(fl_steer, fr_steer, fl_drive, fr_drive, length, width)
form.show()
app.exec()
sim.stopSimulation()
```

Explanation

This program features a GUI object that will appear on the user's screen when the program starts. The user has the option to go forward and backwards, and turn the front wheels to the left or right. The user may also stop the vehicle, setting the vehicle's velocity to 0, and also start and stop the simulation.

Our program features a max speed and turning angle, with lines perpendicular to the wheels on the Z-axis, to better visualise the Ackermann's formula. This formula is utilized when the user wants to turn left or right with the front wheels, in the turn() function. This function receives the desired turning angle, wheel base, and the track width of the car (wheelbase and track width measurements are taken manually) and computes these values into the Ackermann formula (see below) to return the supposed angles of the left and right wheel.

$$\tan \Phi_i = \frac{l}{r-d} = \frac{l}{\frac{l}{\tan\Phi}-d} = \frac{l \tan \Phi}{l-d \tan \Phi}$$

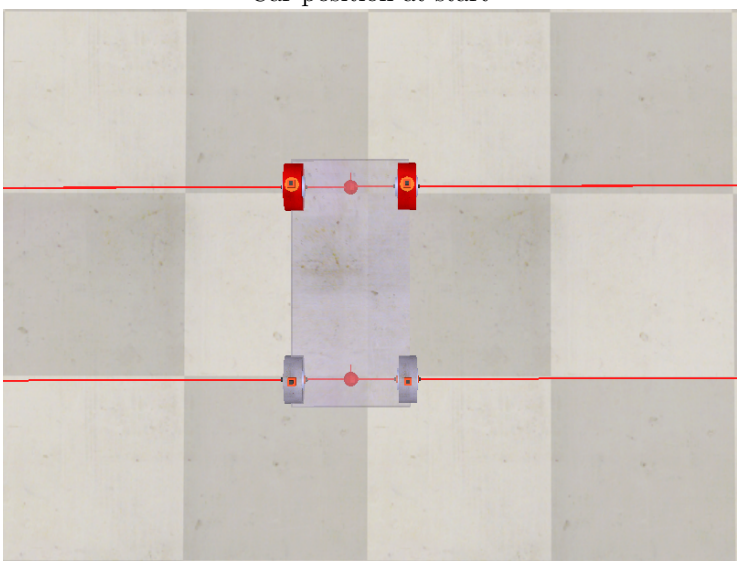
$$\tan \Phi_o = \frac{l}{r+d} = \frac{l}{\frac{l}{\tan\Phi}+d} = \frac{l \tan \Phi}{l+d \tan \Phi}$$

ϕi represents the supposed angle for the left wheel, ϕo represents the supposed angle for the right wheel.

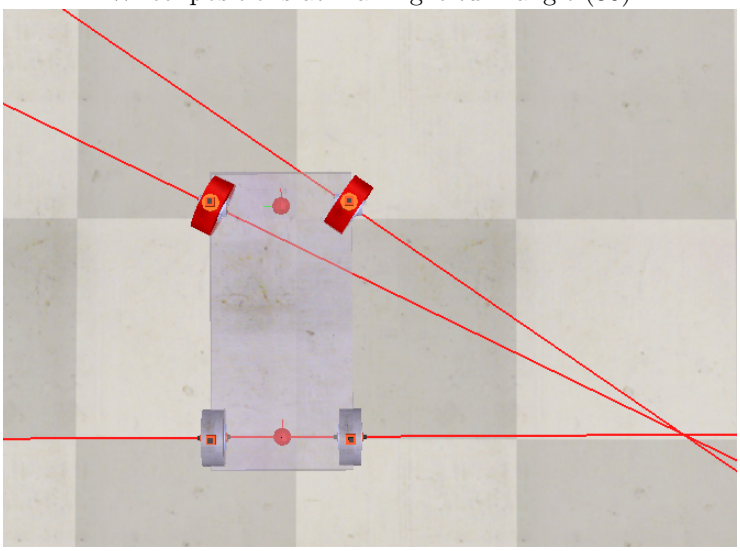
This change in angles is then shown in the CoppeliaSim on the front wheels. With this formula, the car is able to make smooth turns and go in circles, forward and backwards.

Screenshots

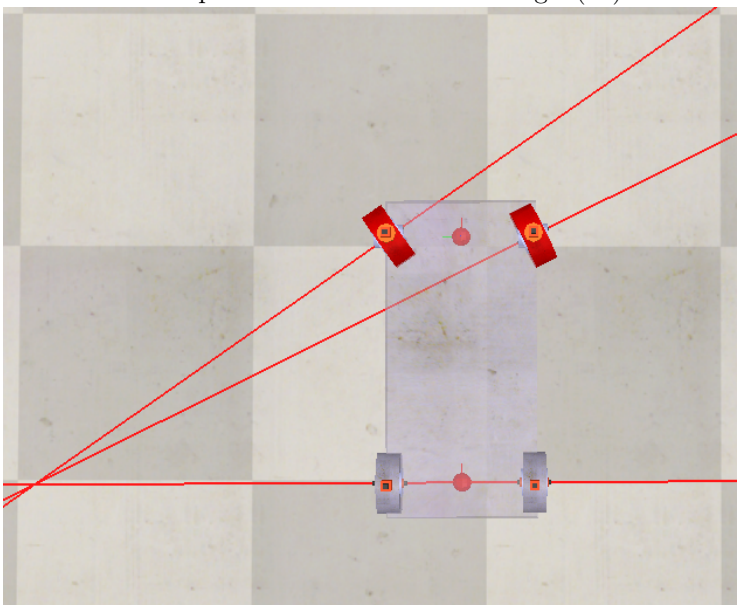
Car position at start



Wheel positions at max right turn angle (30)



Wheel positions at max left turn angle (30)



Engine used is Newton, alternatives such as Bullet 2.83 also accepted. The default engine, Bullet 2.78, is not recommended as this will cause the car to wobble, while the simulation runs, causing inconsistent readings.

Criteria:
1. The front wheel lines must both only intersect on the bottom wheel line (see image 2 and 3) at all times, except when turning angle = 0.
2. A GUI is to be made, to control the movement of the car, and the rotation of the front wheels, at a max turning angle of 30˚.